RÉPUBLIQUE TOGOLAISE

Ministère de l'Économie et des Finances

SIGPÉ

Système Intégré de Gestion du Patrimoine de l'État

Rapport technique complet

Architecture · 13 microservices · Contrats · Infrastructure · Frontend

Version des référentiels : CDC v1.0 (2026-11-11) · Catalogue technique v1.1 (2026-11-12)

Document généré pour la revue de projet — dépôt `github.com/SOVON-14/sigpe_plate-forme`, branche `main`

Sommaire

1. Vision et périmètre du projet
2. Architecture globale
3. Le processus appliqué (contract-first)
4. Les 13 services backend
5. Notions transverses expliquées
6. Infrastructure et pile de développement
7. Frontend SIGPÉ Web
8. État du projet et reste à faire

1. Vision et périmètre

SIGPÉ est une plateforme nationale et souveraine de gestion du patrimoine de l'État togolais : inventaire des biens, affectations, mouvements, maintenance, finance, documents et audit — le tout dans un système distribué, traçable et sécurisé.

1.1 Objectifs

- **Un registre national unique** de tous les biens de l'État (immobilisations, matériels, véhicules…), avec leur cycle de vie complet.
- **Traçabilité totale** : chaque action importante laisse une preuve invérifiable — journal d'audit immuable chaîné par hachage.
- **Gouvernance institutionnelle** : chaque agent ne voit et n'agit que dans le périmètre de son institution (cloisonnement ABAC).
- **Interoperabilité contractuelle** : les services communiquent par des contrats versionnés (OpenAPI pour le synchrone, AsyncAPI pour l'asynchrone).
- **Autonomie des services** : 13 microservices déployables indépendamment, chacun propriétaire de sa base de données.

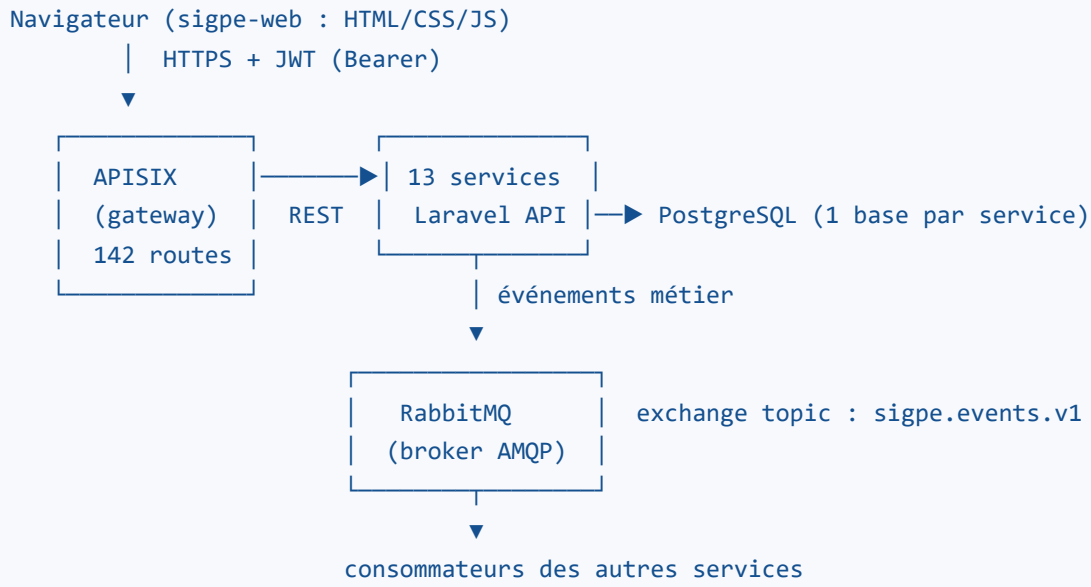
1.2 Les deux documents de référence

Document	Rôle	Version
`SIGPE_Cahier_Des_Charges_v1.0`	**Autorité métier** : règles de gestion, cycles de vie, habilitations, processus institutionnels	v1.0 — 2026-11-11
`SIGPE_Catalogue_Technique_v1.1`	**Autorité technique** : architecture, contrats d'API/ événements, exigences de sécurité et d'audit	v1.1 — 2026-11-12

> **Règle d'or du projet :** le contrat avant le code. Chaque service a d'abord fait l'objet d'un contrat OpenAPI/AsyncAPI relu et validé, puis implémenté en Laravel en respectant strictement ce contrat.

2. Architecture globale

Une architecture microservices événementielle : le navigateur ne parle jamais directement aux services — tout passe par une passerelle API et une couche d'événements.



2.1 Les briques

Brique	Technologie	Rôle
Frontend	HTML/CSS/JS vanilla, nginx	SPA statique, aucune étape de build, auth OIDC PKCE
Passerelle API	Apache APISIX 3.14	142 routes générées automatiquement depuis les OpenAPI ; CORS, proxy vers les services
Services métier	Laravel 13 / PHP 8.5	13 API REST + 13 workers (outbox, consumers)
Bases de données	PostgreSQL (13 bases `sigpe_*`)	Une base par service — aucune jointure inter-services, isolation logique
Broker	RabbitMQ	Échange topic durable `sigpe.events.v1`, files nommées par service
Identité	Keycloak 26	OIDC/OAuth2, realm `sigpe`, client web PKCE + 13 clients services (client_credentials)
Stockage	MinIO (S3)	Documents confidentiels, bucket privé `sigpe-documents`
Orchestration	Docker Compose	34 conteneurs : 13 apps + 13 workers + 8 briques d'infrastructure

2.2 Principe d'isolation

> Chaque microservice est **autonome** : sa propre base, ses propres règles, ses propres événements. Un service ne lit jamais la base d'un autre — il communique soit en synchrone via l'API REST (ex. Movement interroge Asset pour le rattachement TEC-017), soit en asynchrone via un événement RabbitMQ (ex. Acquisition consomme `workflow.instance_completed`).

3. Le processus appliqué

Un enchaînement strict et répétable pour chacun des 13 services.

3.1 Les 5 étapes par service

1. **Contrat (branche `contracts`)** — rédaction de l'OpenAPI 3.1 (endpoints REST), de l'AsyncAPI (événements produits/consommés) et des schémas JSON Schema draft 2020-12 pour chaque événement. Validation par Redocly / AsyncAPI CLI.
2. **Domaine (branche `service/<nom>`)** — enums PHP, migrations par table métier, modèles Eloquent, service métier (logique des cycles de vie et des transitions d'état).
3. **Socle transverse** — recopie du kit commun : middleware JWT Keycloak, RBAC + ABAC institutionnel, idempotence, outbox/inbox, publisher/consumer RabbitMQ, erreurs RFC 9457.
4. **Couche HTTP** — resources JSON, Form Requests de validation, contrôleurs, routes `/api/v1` avec middlewares de permission.
5. **Tests** — tests feature PHPUnit (SQLite en mémoire) couvrant les transitions, la sécurité (401/403), l'idempotence et les flux d'événements. Puis Pint (formatage) et commit.

3.2 Git : le workflow réellement appliqué

Branche	Contenu
`contracts`	14 OpenAPI + 14 AsyncAPI + 72 schémas JSON — figés avant implémentation
`service/<nom>` ×13	Une branche par service, un commit (parfois plusieurs) par service
`main`	Toutes les branches fusionnées — 14 merges, **1 seul conflit** (modèle outbox d'Organization, résolu en gardant la version complète)
`develop`	Synchronisée (fast-forward) sur `main`

> **Attribution :** l'ensemble des commits est signé de l'identité du porteur du projet. Aucune mention d'outil ni de co-auteur n'apparaît dans l'historique.

4. Les 13 services backend

Pour chaque service : ce qu'il fait, son cycle de vie principal et les événements qu'il produit ou consomme.

1 · Identity Service — identité

Rôle : la source de vérité sur les comptes utilisateurs, les délégations de pouvoir et les habilitations.

Métier : cycle de vie des comptes (création, suspension, réactivation), délégations temporaires de signature, calcul des permissions effectives d'un utilisateur (RBAC + règles ABAC).

Produit : `identity.account_created.v1` `identity.account_status_changed.v1` `identity.delegation_created.v1`

Consomme : — (service racine du graphe)

2 · Organization Service — organisation

Rôle : le référentiel institutionnel — qui existe, où, sous quelle forme.

Métier : institutions (administrations centrales, ministères), types d'unités (nationaux + locaux), unités organisationnelles en arbre (directions, services…), postes et responsables. Transitions d'état (active/suspendue/désactivée) tracées avec motif.

Produit : `organization.institution_*` `organization.unit_*`

3 · Asset Service — registre maître

Rôle : le registre maître des biens — la donnée de référence de toute la plateforme.

Métier : fiches actifs (catégorie, sous-catégorie, état physique, statut patrimonial), rattachement TEC-017 (qui détient le bien, depuis quand, où), historique des modifications. Endpoint machine-à-machine `POST /assets/{id}/rattachement` réservé à Movement.

Produit : `asset.asset_created.v1` `asset.status_changed.v1` `asset.rattachement_confirmed.v1`

Consomme : `movement.operation_completed.v1` (confirmation de garde)

4 · Movement Service

mouvements

Rôle : tout ce qui fait bouger un bien : affectations, transferts, prêts, mouvements physiques, droits d'usage partagés.

Métier : opérations de mouvement avec machine à états complète (brouillon → validation workflow → départ → réception → retour/incident) ; gardes (custody) qui disent « ce bien est entre les mains de X » ; droits partagés temporaires. Le rattachement TEC-017 est bouclé : Movement demande à Asset de confirmer la détention.

Produit : `movement.operation_*` `movement.custody_*` `movement.shared_right_*`

Consomme : `workflow.instance_completed.v1`, événements Asset

5 · Inventory Service

inventaires

Rôle : les campagnes d'inventaire physique — la photo réelle du patrimoine sur le terrain.

Métier : campagnes par institution et par période (général annuel, périodique, exceptionnel), agents affectés, constats terrain (bien trouvé / non trouvé / anomalie : endommagé, mal localisé, non enregistré, mal identifié, utilisation non déclarée…), puis vérification contradictoire de chaque constat (confirmé / rejeté / résolu) et rapport d'écarts à la clôture.

Produit : `inventory.campaign_*` `inventory.finding_*`

6 · Workflow Service

circuits

Rôle : le moteur de circuits de validation — l'instance centrale d'approbation de la plateforme.

Métier : templates de workflows nationaux ou par institution (étapes groupées, mode « tous » ou « un seul », SLA, étape nationale obligatoire), instances liées à une opération (acquisition, mouvement, sortie…), tâches à décider (approuver / rejeter / demander correction, commentaire obligatoire hors approbation), délégations temporaires, figeage de la version du circuit au démarrage.

Produit : `workflow.instance_completed.v1` `workflow.instance_cancelled.v1`

Consomme : événements des services demandeurs

7 · Maintenance Service

maintenance

Rôle : la maintenance du patrimoine — demandes, plans, interventions, coûts, prestataires, stocks de pièces.

Métier : demandes corrective / préventive / conditionnelle (voie urgente justifiée), cycle draft → validation → planification → intervention (start / interrupt / complete / cancel) avec coûts par catégorie (main-d'œuvre, pièces, consommables, transport, externe) ; prestataires évalués sur 6 critères ; pièces de rechange, entrepôts, mouvements de stock, seuils de réapprovisionnement avec alerte automatique.

Produit : `maintenance.request_*` `maintenance.intervention_*` `maintenance.spare_part.below_threshold.v1` (+ 6 autres)

Consomme : `workflow.instance_completed.v1`

8 · Acquisition Service

acquisitions

Rôle : le cycle complet d'achat public : du besoin à l'enregistrement du bien dans le registre Asset.

Métier : demandes avec lignes d'articles (quantité, unité, valeur estimée justifiée), soumission au workflow, sélection de l'offre (fournisseur + justification), commande, livraisons, réceptions ligne par ligne avec numéros de série → chaque bien accepté est enregistré dans Asset via un appel TEC authentifié ; clôture impossible tant que des acceptations ne sont pas enregistrées. Séquence métier `ACQ-AAAA-NNNNNN`.

Produit : `acquisition.request_*` `acquisition.delivery_*` `acquisition.reception_*`

Consomme : `workflow.instance_completed.v1` ; appelle `POST /assets` sur Asset

9 · Audit Service

audit

Rôle : le garant de la confiance — un journal immuable de tout ce qui s'est passé.

Métier : consomme **tous** les événements de la plateforme (binding wildcard `#`) et les transforme en entrées d'audit ; chaîne de hachage SHA-256 vérifiable en un appel ; classification de sensibilité (normal / sensible / critique) ; chaque vérification de chaîne émet elle-même un événement — jamais de contrôle silencieux.

Produit : `audit.entry_recorded.v1` `audit.integrity_check_completed.v1`

API : liste filtrable (8 critères), détail complet, `GET /audit-entries/chain/verify`, statistiques — permissions `audit.read` / `audit.verify`

10 · Document Service

documents

Rôle : le dépôt documentaire national — contrats, actes, pièces justificatives.

Métier : 8 catégories nationales, 4 niveaux de confidentialité (public administratif → hautement sensible), versions **immuables** (jamais d'écrasement), contrôle format + empreinte SHA-256, stockage privé S3/MinIO, cycle de vie (brouillon → validé → archivé), demandes d'accès exceptionnelles avec décision motivée et journal de chaque consultation/téléchargement.

Produit : `document.*` (création, version, validation, accès)

11 · Notification Service

notifications

Rôle : le service d'alerte — transformer les événements en messages utiles.

Métier : 3 canaux (in-app, email, SMS), préférences par type d'événement et par utilisateur, notifications **critiques obligatoires** (non désactivables), accusé de prise en compte *distinct* de l'approbation métier, relances programmées.

Produit : `notification.sent.v1` `notification.acknowledged.v1`

Consomme : la plupart des événements métier (alertes stock, anomalies, validations…)

12 · Finance Service

finance

Rôle : la valorisation et la sortie du patrimoine — ce que valent les biens et comment ils quittent le domaine.

Métier : valorisations (acquisition, actuelle, nette comptable, remplacement) avec amortissement (linéaire, dégressif) et valeur explicitement « indéterminée » quand elle l'est ; coûts patrimoniaux par catégorie ; dossiers de sortie définitive (réforme, vente, destruction, perte, vol, don) à 8 statuts avec **niveau de compétence requis** (institutionnel / ministériel / national), enquête obligatoire pour perte/vol — le bien n'est jamais retiré automatiquement.

Produit : `finance.valuation_*` `finance.disposal_*`

13 · Reporting Service

reporting

Rôle : les tableaux de bord et rapports nationaux — en lecture seule, alimentés par les événements.

Métier : ingère les événements et maintient des **projections** (état courant des actifs, compteurs par domaine) ; indicateurs par scope national/institutionnel ; modèles de rapports nationaux exécutables par période avec paramètres ; exécutions historisées et exportables ; suivi du lag de synchronisation.

Produit : — (service terminal de lecture)

Consomme : la quasi-totalité des événements métier

5. Notions transverses — expliquées simplement

Les concepts qui reviennent dans tout le backend, avec leur justification métier.

Outbox (boîte d'envoi transactionnelle)

Quand un service modifie sa base *et* doit notifier les autres, il écrit l'événement **dans la même transaction SQL** que la donnée métier (table `outbox_messages`). Un worker (`sigpe:outbox:publish`) publie ensuite sur RabbitMQ.

Pourquoi : impossible de perdre un événement si le service plante entre « j'ai enregistré » et « j'ai notifié ». C'est la garantie *au moins une fois*.

Inbox (boîte de réception dédoublonnée)

Table `inbox_events` : chaque événement consommé est enregistré par son `event_id` avant traitement. Si RabbitMQ le livre deux fois (at-least-once), le second est ignoré.

Pourquoi : les brokers distribuent « au moins une fois » — jamais « exactement une fois ». L'inbox rend la consommation *idempotente*.

Idempotence (en-tête `Idempotency-Key`)

Toute mutation REST exige une clé unique. Si le client réessaie (timeout réseau), la même clé renvoie la **même réponse** au lieu de créer un doublon. Clé identique + payload différent → erreur 409.

JWT Bearer + Keycloak

Le navigateur obtient un jeton OIDC (Authorization Code + PKCE) auprès de Keycloak. Les services valident sa signature via le JWKS et lisent les claims : `sub` (qui), `permissions` (quoi), `institution_id` (où). Les appels machine-à-machine (Movement→Asset) utilisent des clients `client_credentials`.

RBAC + ABAC

RBAC (Role-Based) : la permission `movement.operations.manage` autorise l'action. **ABAC** (Attribute-Based) : la ressource porte un `institution_id` et l'utilisateur aussi — un agent de l'institution A reçoit 403 sur les ressources de l'institution B, même avec la bonne permission. L'auditeur national (sans institution) voit tout.

Machine à états (state machine)

Chaque cycle de vie est une liste fermée de transitions : une opération de mouvement ne peut passer à `completed` que depuis les états prévus, avec les préconditions métier (workflow approuvé, TEC-017 confirmé…). Une transition interdite → 409.

Chaîne de hachage (audit)

Chaque entrée d'audit contient `entry_hash = SHA-256(previous_hash + contenu)`. Modifier ou supprimer une entrée rompt tous les maillons suivants — détectable par `GET /audit-entries/chain/verify`. C'est l'équivalent logiciel du « livre de comptes » : inviolable par conception.

RFC 9457 (Problem Details)

Format standard d'erreur `application/problem+json` : `type`, `title`, `status`, `detail`, `instance`. Le frontend l'affiche tel quel — une seule convention d'erreur pour toute la plateforme.

UUIDv7

Identifiants uniques *triables par date* (timestamp + aléatoire) — indexables en base sans fragmenter les pages, générables hors-ligne sans coordination.

Séquences métier

Références lisibles par les humains en parallèle des UUID : `ACQ-2026-000123` (acquisitions), format équivalent côté autres services — ce que l'on cite dans les courriers officiels.

Single Database per Service

Chaque service possède sa base PostgreSQL (`sigpe_asset` , `sigpe_movement` ...). Conséquence : scalabilité indépendante, pas de couplage transactionnel — les cohérences inter-services passent par les événements et les API.

Outbox → RabbitMQ → Inbox (le trio de fiabilité)

La chaîne complète : la donnée et l'événement naissent dans la même transaction (outbox) → le publisher garantit la livraison (publisher confirms) → le consommateur dédoublonne (inbox) → la donnée est traitée. C'est le pattern qui rend un système distribué *fiable sans transaction distribuée*.

6. Infrastructure et pile de développement

6.1 Conteneurs (docker compose)

Groupe	Conteneurs
Applications	13 services Laravel (`php artisan migrate --force && artisan serve`)
Workers	13 workers : `queue:work` + boucle `sigpe:outbox:publish` , + `sigpe:events:consume` pour les 7 services consommateurs
Données	PostgreSQL (init : 13 bases + users), MinIO + init bucket
Messagerie	RabbitMQ + conteneur one-shot d'init (vhost, user, exchange)
Sécurité	Keycloak (realm importé : client web PKCE, 13 clients services, client scope claims `permissions` / `institution_id`)
Passerelle	APISIX standalone, `apisix.yaml` généré depuis les OpenAPI (142 routes)
Frontend	`sigpe-web` : nginx statique sur le port 8088

6.2 Points techniques notables

- **Génération APISIX :** `gateway/apisix/generate-routes.php` lit les 13 OpenAPI et produit les routes — jamais édité à la main. Un conflit de route (`/assets/{id}` Asset vs Reporting) a été résolu par un chemin dédié `/api/v1/reporting/assets/{id}` .
- **Génération Compose :** `infrastructure/docker/generate-compose-services.php` produit les 26 services applicatifs de façon idempotente.
- **Secrets :** aucun en dur — 13 `SIGPE_KC_SECRET_*` + mots de passe via `.env` (gitignoré), documentés dans `.env.example` .
- **JWT inter-réseaux :** issuer = URL publique Keycloak, JWKS fetché en interne (`keycloak:8080`) — les services valident sans exposer le réseau interne.

7. Frontend SIGPÉ Web

SPA 100 % HTML/CSS/JS vanilla — zéro framework, zéro build, servie par nginx. Choix assumé : lisibilité, auditabilité, absence de dépendances npm.

7.1 Socle

Fichier	Responsabilité
`js/auth.js`	Keycloak Authorization Code + PKCE (S256, state, refresh token, logout)
`js/api.js`	Client fetch : Bearer, refresh automatique, `Idempotency-Key` UUID sur mutations, parsing RFC 9457, upload multipart
`js/ui.js`	~25 icônes SVG, toasts, modals (avec `onOpen`), badges de statut, squelettes, pagination
`js/app.js`	Routage hash, layout sidebar/topbar, **filtrage des modules par permissions JWT**
`css/app.css`	Design system : tokens navy/or institutionnels, KPI cards, tables responsives, `prefers-reduced-motion`, focus visibles

7.2 Les 13 modules

Dashboard · Registre des actifs · Notifications · Mouvements (opérations/gardes/droits partagés, machine à états complète) · Maintenance · Inventaires · Documents (upload multipart, versions, accès) · Finance (valorisations, coûts, sorties) · Acquisitions (cycle complet) · Circuits de validation (instances + décisions de tâches) · Journal d'audit (filtres, détail, **vérification de chaîne en un clic**) · Rapports & indicateurs · Organisation.

7.3 Alignement contractuel vérifié

Chaque vue a été confrontée aux routes réelles des services (`routes/api.php`) — deux écarts détectés et corrigés : la liste globale des constats d'inventaire (scopée par campagne) et les demandes d'accès documents (scopées par document, décision `approved` / `denied` + motif obligatoire).

8. État du projet et reste à faire

8.1 Ce qui est fait et vérifié

- 13/13 services implémentés, ~200 tests feature verts, Pint appliqué partout
- 14 branches fusionnées sur `main`, `develop` synchronisée, 1 conflit résolu
- Contrats validés : 14 OpenAPI 3.1, 14 AsyncAPI, 72 schémas JSON
- Pile Docker complète générée et validée syntaxiquement (`docker compose config`)
- Frontend complet : 13 modules, syntaxe JS validée (`node --check`)
- Poussé sur GitHub — `main` synchronisé avec `origin/main`

8.2 Ce qui reste

#	Tâche	Blocage / note
1	Validation E2E de la pile Docker	Docker Desktop à redémarrer (ou machine alternative) — `docker compose up -d --build`
2	Utilisateur de test + permissions dans Keycloak	Requis pour naviguer dans le frontend (sinon 403)
3	Tests des flux inter-services réels	AMQP, TEC-017, Workflow→métier — jamais validés en conditions réelles
4	Observabilité	Prometheus, Grafana, OpenTelemetry, Loki, Tempo — non commencé
5	CI/CD	Pipelines — non commencé
6	Seeders / données de démo	Aucun jeu de données — écrans vides au premier démarrage
7	Approfondissements frontend (optionnel)	Plans/prestataires/entrepôts (Maintenance), historique par bien (Mouvements), journal d'accès (Documents)

8.3 Prochaine étape recommandée

> Sur une machine où Docker fonctionne : cloner le dépôt, renseigner `.env` , `docker compose up -d --build` , créer un utilisateur dans le realm Keycloak `sigpe` , puis ouvrir `http://localhost:8088` . Les erreurs révélées par ce premier run conditionneront les derniers ajustements avant la mise en production.
